

# MarketLab

## Engineering Standards & Forward Roadmap

Prepared for: Builder / Development Team — Prepared by: Michael

Purpose of this document: MarketLab is being built to demonstrate software engineering strength for **Quant Software Engineering (Quant SWE)** recruiting at large quantitative and trading firms — not quant research or trading roles. This document sets the engineering bar the project must meet, the product improvements needed to make it something people will actually want to use, and a single prioritized roadmap that reconciles both. It is written to be handed directly to the builder as a working brief.

### 1. Positioning & Audience

---

**Primary goal:** Produce evidence, for resume screens and interviews at Quant SWE roles, that Michael builds correct, reliable, observable distributed systems — not that he has found a profitable trading signal. The system-design and correctness story matters more than research depth or strategy sophistication.

**Secondary goal:** Make the product something a real user (student, investment club member, or developer learning financial research) would actually choose to use, because a system nobody uses is weaker recruiting evidence than one with real, if small, adoption.

**What this means in practice:** every engineering hour should be justified against one of these two goals. Feature work that doesn't strengthen either the correctness/reliability story or genuine user value should be deprioritized, however interesting it is to build.

### 2. Current State (Summary)

---

A private, deployed web application exists with real functionality: portfolio accounting (deposits, fees, dividends, splits, gains), a single moving-average backtesting strategy compared against buy-and-hold, an Alpha Vantage data connector plus CSV import, saved/frozen experiment records with JSON/CSV export, and a separate Python program that independently re-checks calculations. A background job service (Node.js, MongoDB, worker threads) and a C++ risk-statistics/performance component are implemented and locally verified but **not yet activated on the live site**. The latest release passed 14 focused research tests, type checking, and a production build.

**Read of this state:** the accounting and reproducibility foundation is genuinely strong. The two biggest gaps relative to the Quant SWE bar are (a) correctness and reliability claims that are described but not yet demonstrated under failure/concurrency, and (b) a product experience that still requires the builder to walk a user through setup.

## 3. Engineering Standards to Meet (Quant SWE Bar)

---

These are the standards a Quant SWE interviewer/recruiter will implicitly check for. Each should be **demonstrated with a test or measurement**, not just described in prose.

### 3.1 Concurrency correctness

Identify at least one real race condition in the system and fix it with an explicit mechanism (idempotency key, optimistic locking, database transaction). Candidates: two simultaneous writes to the same saved experiment; two background workers picking up the same job. Write a test that fails without the fix and passes with it. This is the single highest-value standard for this audience.

### 3.2 Failure recovery

Demonstrate, not just claim, that a background job survives a worker crash mid-execution and resumes exactly once — no duplicate result, no lost job. A recorded test or short demo of killing a worker process and showing correct recovery is strong evidence.

### 3.3 Layered, labeled test suite

Restructure tests into clearly named layers instead of one flat count:

- **Unit tests** on financial invariants (e.g., conservation of money across every transaction type; no operation may silently create or destroy value).
- **Integration tests** on the API layer (request validation, error handling, persistence).
- **Cross-implementation checks**: the Python vs. TypeScript agreement check should run as a formal CI test, not just an ad-hoc tool.

### 3.4 Measured performance, not qualitative claims

Produce at least one real, honestly-reported load number: concurrent backtest requests handled, P99 latency, throughput of the C++ risk component on a defined workload. Approximate numbers, honestly labeled as such, are fine — the point is that they are measured rather than asserted.

### 3.5 Money-correctness as an explicit design choice

Exact/decimal arithmetic for all monetary calculations is already a good choice — make it visible: one README paragraph explaining why (floating-point drift is a well-known bug class in financial systems), backed by a test that would fail under naive floating-point arithmetic.

### 3.6 Background service activated, at least in staging

Until the Node/MongoDB/worker-thread service and the C++ component run somewhere reachable (even a staging environment), "processes jobs safely" remains a design claim rather than an observed behavior. This is more work than 3.1–3.5, so it is sequenced later, but it is the most convincing possible proof of the distributed-systems story.

## 4. Product Improvements (Make People Want to Use It)

---

The differentiator is trust and transparency, not more strategies or more indicators. The improvements below make that differentiator visible and low-friction instead of buried in methodology.

### 4.1 Zero-setup demo moment

A one-click demo using a realistic, clearly-labeled synthetic portfolio, so a new visitor reaches a genuine "I didn't realize fees ate that much of my return" moment in under a minute, with no import required. Highest-leverage single change: most visitors never get past setup friction today.

### 4.2 Visible "confidence certificate" on every result

Surface, at the top of every report: data source and completeness, what was independently verified, and what cost/assumption choices were made. This turns the existing reproducibility engineering into a visible feature instead of hidden methodology — and it is the one competitors are unlikely to have.

### 4.3 Plain-language result summaries

Auto-generate a one- or two-sentence human summary alongside every metrics table, e.g.: "Your rule beat buy-and-hold before costs, but lost after fees — mostly because it traded 40 times in 6 months." Most users do not parse a Sharpe ratio; they understand a sentence.

### 4.4 Shareable, read-only report links

Let a saved experiment be shared as a view-only link that carries its confidence certificate with it. This is the mechanism by which the product could actually spread — someone shows a result to a friend or their investment club.

### 4.5 Real brokerage import formats

Support Fidelity/Schwab/Robinhood export formats directly instead of requiring a generic CSV. This removes the single biggest practical barrier for the stated target users (students, investment clubs) and is likely the highest-value unglamorous engineering hour available.

### 4.6 Narrative timeline on the equity curve

Overlay actual events — a deposit, a dividend, a split, a trade — directly on the return chart, so "why did my portfolio move" is visual instead of requiring cross-referencing a separate table.

### 4.7 Lightweight collaboration for investment clubs

Allow a saved experiment to be commented on or forked by a second person. This turns a solo tool into something a club actually uses together, which is a real (if small-scale) retention mechanism.

### Deliberately deprioritized for "wow"

More strategies, more asset classes, more indicators, and statistical-significance testing on backtests. These make the product feel like every other backtester and do not serve either the recruiting story or the trust/transparency differentiator. They can return later as research-depth work, but not now.

## 5. Single Prioritized Roadmap

This reconciles the engineering standards (Section 3) and the product improvements (Section 4) into one ordered list. Earlier items are higher priority; later items depend on earlier ones being solid.

| #  | Milestone                                                      | Why it matters                                       | Done when...                                                   |
|----|----------------------------------------------------------------|------------------------------------------------------|----------------------------------------------------------------|
| 1  | Fix one real concurrency bug + test                            | Core Quant SWE correctness story                     | Test fails without fix, passes with fix                        |
| 2  | Demonstrate worker-failure recovery                            | Distributed-systems credibility                      | Killed worker mid-job; job resumes exactly once                |
| 3  | Layered, labeled test suite (unit / integration / show engine) | Shows engineering discipline, not just a test script | Suite is split, invariant tests exist, cross-check runs in CI  |
| 4  | Zero-setup demo experience                                     | Removes the #1 barrier to anyone seeing this value   | Any visitor sees a real insight in under a minute, no import   |
| 5  | Visible confidence certificate + plain-language summary        | Makes the differentiator visible                     | Every report shows verification status and a one-line summary  |
| 6  | Real brokerage CSV formats                                     | Removes the biggest adoption barrier                 | Fidelity/Schwab/Robinhood exports import without manual clean  |
| 7  | One measured load/performance number                           | Replaces qualitative claims with evidence            | P99 latency or throughput reported for a defined workload      |
| 8  | Money-correctness README paragraph + claim-flight-test         | Claims flight-test correctness evidence              | Paragraph written; test demonstrates the float bug class       |
| 9  | Shareable read-only report links                               | Distribution / word-of-mouth mechanism               | Any saved experiment can be viewed via a link, unauthenticated |
| 10 | Narrative timeline overlay on equity curve                     | Improves comprehension of results                    | Events are plotted directly on the return chart                |
| 11 | Activate background service in staging                         | Converts design claim into observed behavior         | Restart and recover in a real (non-local) environment          |
| 12 | Lightweight collaboration (comment/fork or review experiments) | Retention mechanism for the club use case            | A second user can view, comment, or fork a shared experiment   |

Items 1–3 are the highest-value work for the recruiting goal and require no new product surface. Items 4–6 are the highest-value work for real user adoption. Items 7–12 round out the story once the first two clusters are solid.

## 6. Direct Brief to the Builder

---

The following can be copied directly as a working instruction set.

1. Find and fix one real concurrency bug (concurrent writes to a saved experiment, or duplicate job pickup by two workers). Use an idempotency key or optimistic locking. Write a test that fails before the fix and passes after.
2. Simulate a worker crash mid-job in a test or recorded demo. Confirm the job resumes exactly once, with no duplicate or lost result.
3. Reorganize the test suite into unit tests (financial invariants), integration tests (API layer), and the Python/TypeScript cross-check (run as a formal CI test).
4. Build a one-click demo portfolio (clearly labeled synthetic data) that shows a genuine insight with zero import required.
5. Add a visible "confidence certificate" summary block to every report: data source, verification status, assumptions made.
6. Generate a one-to-two sentence plain-language summary alongside every backtest/portfolio result.
7. Add direct import support for at least one real brokerage export format (Fidelity, Schwab, or Robinhood).
8. Produce one honestly-reported load/performance measurement (API latency under concurrent requests, or C++ component throughput on a defined workload).
9. Add a README paragraph on why exact/decimal arithmetic is used for money, backed by a test that would fail under naive floats.
10. Implement shareable, read-only links for saved experiments.
11. Add an events-on-timeline overlay to the equity curve chart.
12. Once the above is solid, get the background job service running in at least a staging environment.

**Explicitly hold off on:** additional trading strategies, broader asset-class or data coverage, and statistical-significance testing on backtests. These serve a quant-research story, not the Quant SWE story this project is being built to support, and they compete for time with the items above.

## 7. Resume Framing (If Built to This Standard)

---

If the milestones in Sections 3–5 are completed, this is roughly how the project should read on a resume. Note what is deliberately absent: strategy names, Sharpe ratios, indicator counts. That framing belongs to a quant-research resume, not a Quant SWE one, and would weaken this section if included.

### **MarketLab — Portfolio Research & Backtesting Platform | Personal Project | [github.com/...](#)**

- Designed and built a full-stack financial research platform with independently-verified calculations, transactional accounting, and reproducible backtests
- Identified and resolved a race condition in concurrent experiment writes using optimistic locking; added a regression test covering the failure mode
- Implemented exactly-once job recovery for a background worker pool (Node.js/worker\_threads) so calculations survive process crashes without duplicating or losing results
- Built a cross-language correctness check: a Python reimplemention independently verifies TypeScript-computed portfolio and strategy results, run as an automated CI gate
- Measured system behavior under load (e.g., "P99 latency of X ms at N concurrent backtest requests") and published the result rather than asserting it
- Used exact decimal arithmetic throughout the accounting layer to eliminate floating-point drift in monetary calculations, with a test proving the failure mode it prevents

**Tech stack line:** TypeScript, React, Node.js, Python, C++, MongoDB, SQLite/D1, worker threads, GitHub Actions (CI)

## 8. A Quant SWE Recruiter's Perspective

---

Framed honestly rather than encouragingly: this project would likely earn a phone screen, not an automatic advance past it — and the project itself is not the deciding factor either way.

### What actually gates a resume screen at a top quant shop

School/GPA, coursework rigor (systems, algorithms, concurrency, sometimes a numerical/quant course), competitive-programming results, referrals, and prior internships. A side project is real evidence in an **interview**, but at most firms it is a tie-breaker or conversation-starter at the resume stage, not the primary signal. If those other boxes are weak, a polished project alone usually does not flip the decision.

### What would raise confidence to a clearer "yes"

1. A live, clickable link — not "deployed private app." Anything requiring setup or trust in a description gets discounted by a recruiter skimming quickly.
2. One hard number inside the resume bullet itself (an actual P99 latency or throughput figure), not just the category "measured performance under load." Vague claims read as unverified even when true.
3. Evidence of algorithmic/CS fundamentals alongside the systems work — a competitive-programming rating, a strong grade in a hard algorithms/systems course, or solved hard problems. Without this, the project alone signals "good software engineer," which is necessary but not sufficient for a Quant SWE screen specifically.
4. A short "what I'd do differently at scale" reflection in the README. Junior candidates describe what they built; stronger candidates critique their own design. This costs little and signals seniority of thinking disproportionately.

### Why the lean is toward "yes" rather than "no"

The concurrency bug fix, the exactly-once recovery, and the independent cross-language verification are the exact things a Quant SWE interviewer asks about in a system-design round — and here they would already be pre-answered with a real story instead of a hypothetical one. That is rarer than expected, even among CS students applying to these roles. Combined with reasonable coursework and no red flags, it is enough to earn a conversation.

### Remaining improvements, in order

1. Ship the live link and rewrite bullets to include hard numbers — costs nothing but attention to presentation.
2. Surface independent evidence of algorithmic strength (competitive programming, a hard-course grade) — the project structurally cannot fill this gap on its own.
3. Activate the background service in staging. "Implemented but not running anywhere" is the one line in the story that invites the follow-up question you don't want: "so has this actually run?"